Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет ИТМО»

Факультет информационных технологий и программирования

ЛАБОРАТОРНАЯ РАБОТА №2

на тему «документирование»

Выполнил: Васютченков К. А.
Группа: M3116

Санкт-Петербург, 2026

Документирование. Лабораторная работа №2

Продолжаем работать с проектом, который был создан в рамках первой лабораторной работы.

Документация является неотъемлемой частью любого ИТ продукта и им обогатим наш проект как отдельной документацией, так и внедрим её в исходный код.

1. Каждое декларирование функции в файлах проекта должно начинаться с блока комментария, в котором описывается сама функция и пример её вызова.

```
def square(n):
    '''Принимает число n, возвращает квадрат числа n'''
    return n**2
```

или

```
def add_binary(a, b):
    '''
    Возвращает сумму двух десятичных чисел в двоичном формате.

            Параметры:
                    a (int): первое десятичное целое число
                    b (int): второе десятичное целое число

            Возвращаемое значение:
                    binary_sum (str): двоичная строка суммы a и b
    '''
```

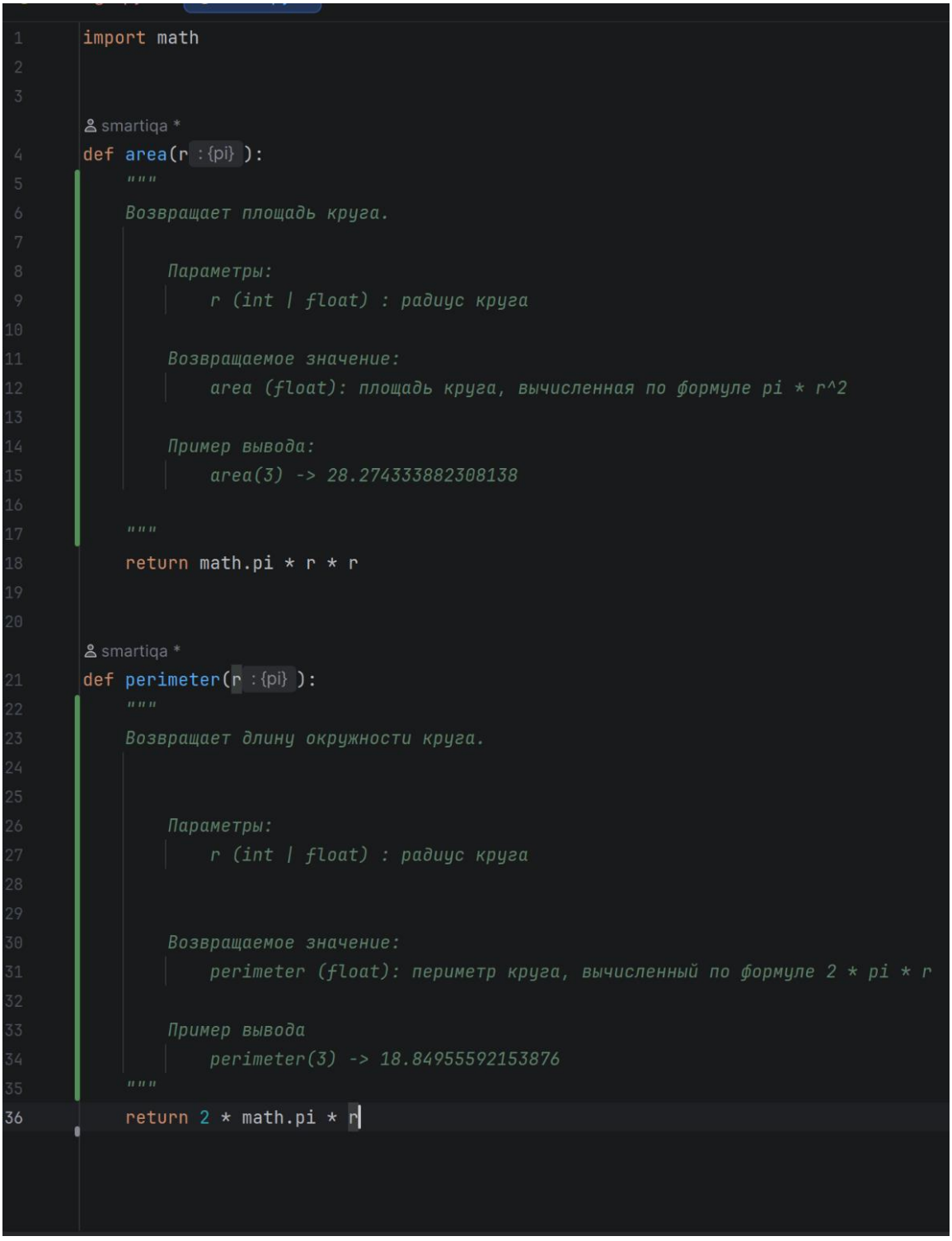
2. В структуре проекта должен появиться каталог docs, в котором должна находиться документация по проекте написанная в нотации markdown (подробнее про неё читаем здесь), включающая в себя следующие разделы:

- общее описание решения
- описание каждой функции с примерами вызова
- история изменения проекта с хешами комитов (кроме последней записи)

Для защиты - нужно будет показать преподавателю свой проект с созданными артефактами.

1) Описываем каждую функцию в 4-х файлах: circle.py, rectangle.py, square.py, triangle.py:

circle.py:



```python
import math


smartiqa *
def area(r :{pi} ):
    """
    Возвращает площадь круга.

        Параметры:
            r (int | float) : радиус круга

        Возвращаемое значение:
            area (float): площадь круга, вычисленная по формуле pi * r^2

        Пример вывода:
            area(3) -> 28.274333882308138

    """
    return math.pi * r * r

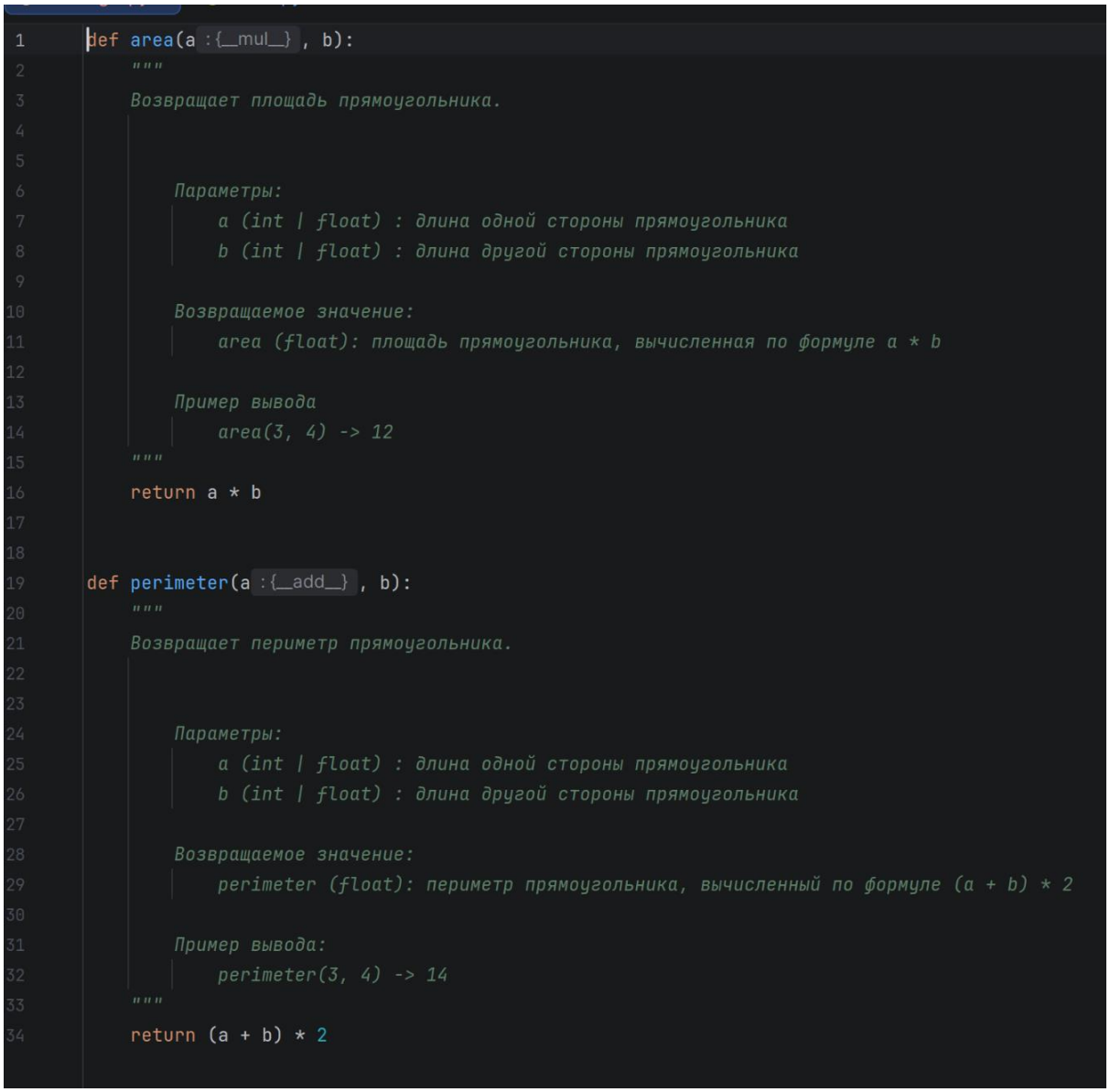

smartiqa *
def perimeter(r :{pi} ):
    """
    Возвращает длину окружности круга.


        Параметры:
            r (int | float) : радиус круга


        Возвращаемое значение:
            perimeter (float): периметр круга, вычисленный по формуле 2 * pi * r

        Пример вывода
            perimeter(3) -> 18.84955592153876
    """
    return 2 * math.pi * r
```

rectangle.py:



```python
def area(a :{__mul__}, b):
    """
    Возвращает площадь прямоугольника.



        Параметры:
            a (int | float) : длина одной стороны прямоугольника
            b (int | float) : длина другой стороны прямоугольника

        Возвращаемое значение:
            area (float): площадь прямоугольника, вычисленная по формуле a * b

        Пример вывода
            area(3, 4) -> 12
    """
    return a * b


def perimeter(a :{__add__}, b):
    """
    Возвращает периметр прямоугольника.

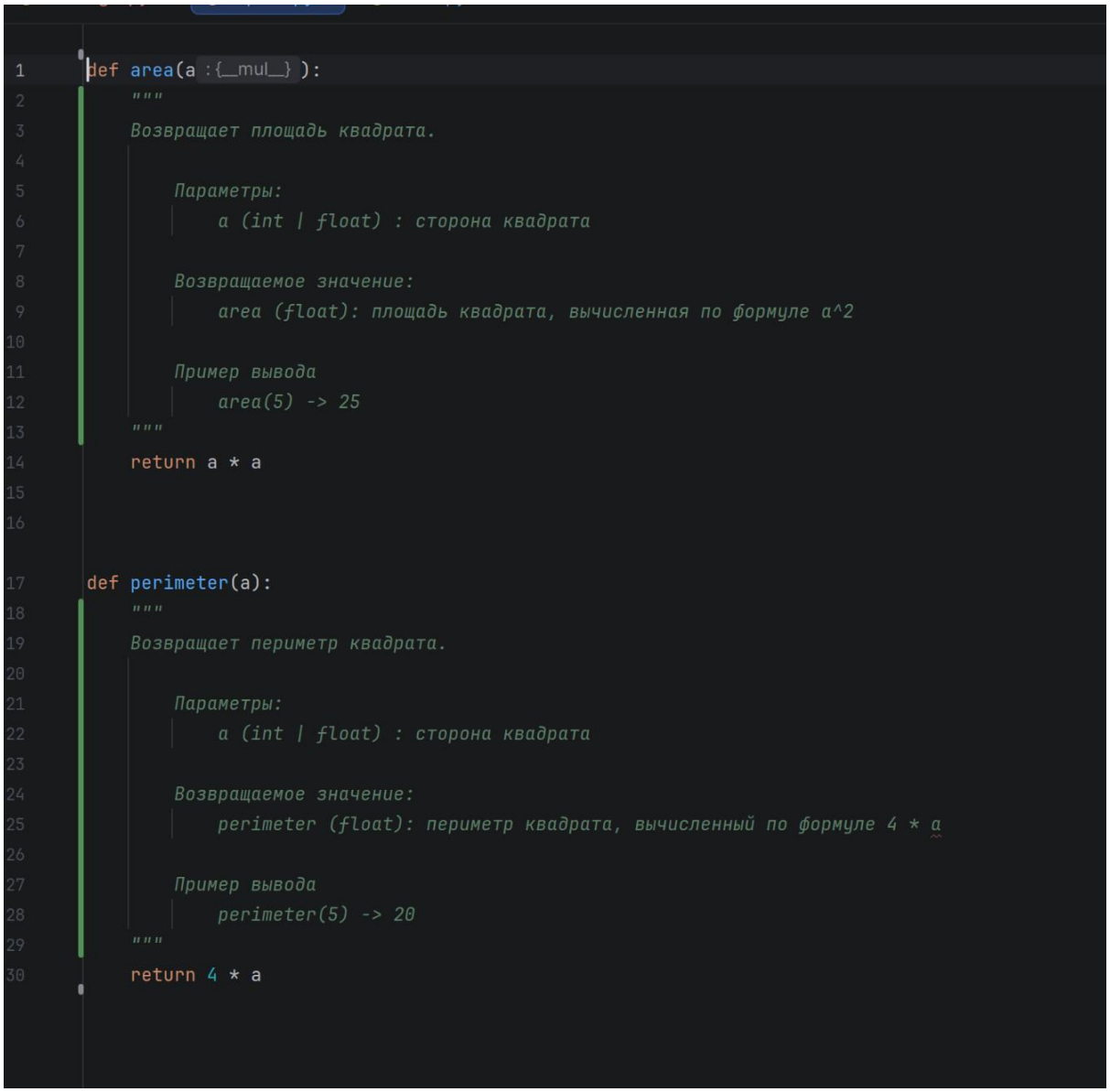


        Параметры:
            a (int | float) : длина одной стороны прямоугольника
            b (int | float) : длина другой стороны прямоугольника

        Возвращаемое значение:
            perimeter (float): периметр прямоугольника, вычисленный по формуле (a + b) * 2

        Пример вывода:
            perimeter(3, 4) -> 14
    """
    return (a + b) * 2
```

square.py



```python
def area(a :{__mul__} ):
    """
    Возвращает площадь квадрата.

        Параметры:
            a (int | float) : сторона квадрата

        Возвращаемое значение:
            area (float): площадь квадрата, вычисленная по формуле a^2

        Пример вывода
            area(5) -> 25
    """
    return a * a



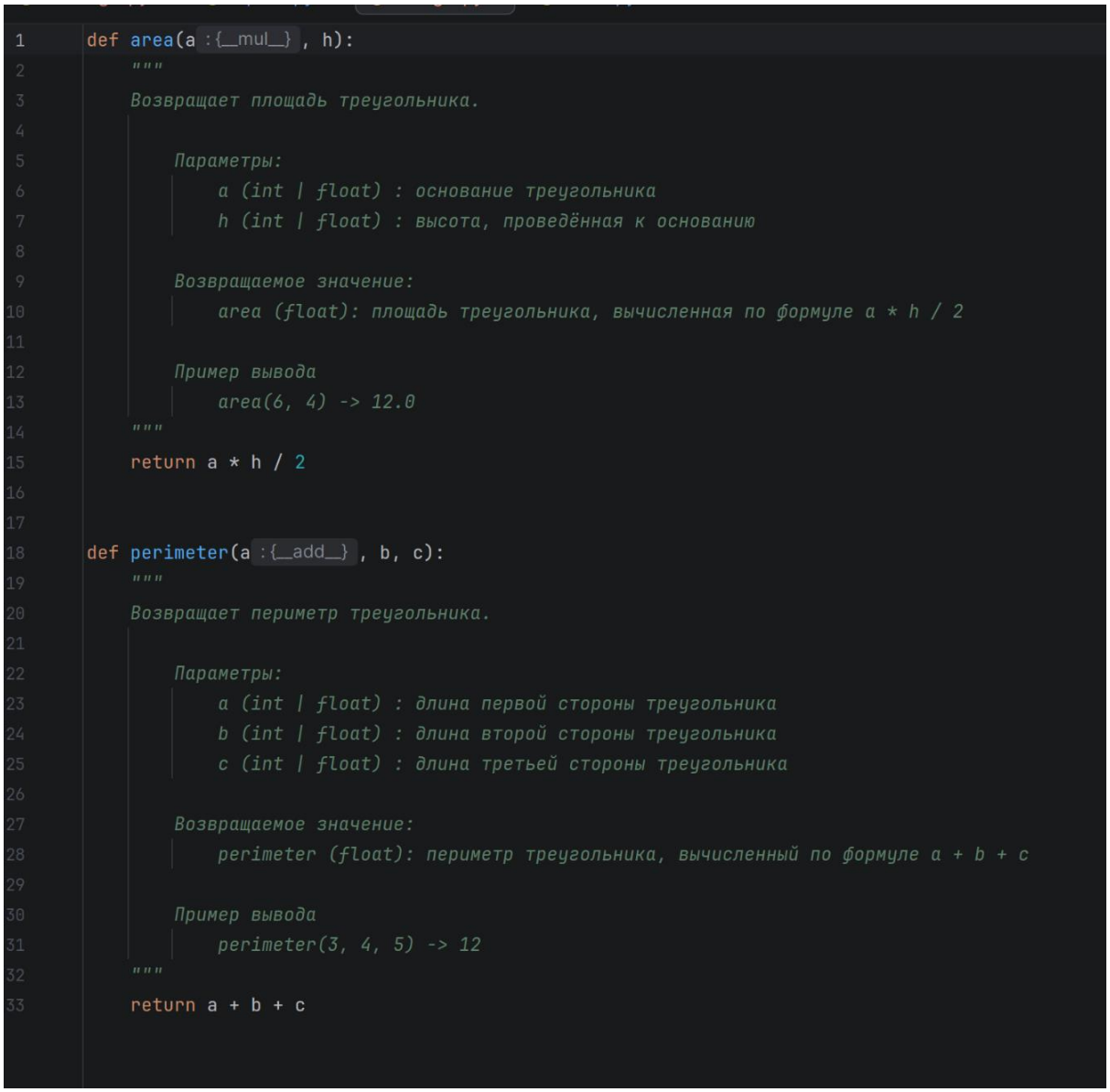
def perimeter(a):
    """
    Возвращает периметр квадрата.

        Параметры:
            a (int | float) : сторона квадрата

        Возвращаемое значение:
            perimeter (float): периметр квадрата, вычисленный по формуле 4 * a

        Пример вывода
            perimeter(5) -> 20
    """
    return 4 * a
```

triangle.py



```python
def area(a :{__mul__}, h):
    """
    Возвращает площадь треугольника.

        Параметры:
            a (int | float) : основание треугольника
            h (int | float) : высота, проведённая к основанию

        Возвращаемое значение:
            area (float): площадь треугольника, вычисленная по формуле a * h / 2

        Пример вывода
            area(6, 4) -> 12.0
    """
    return a * h / 2

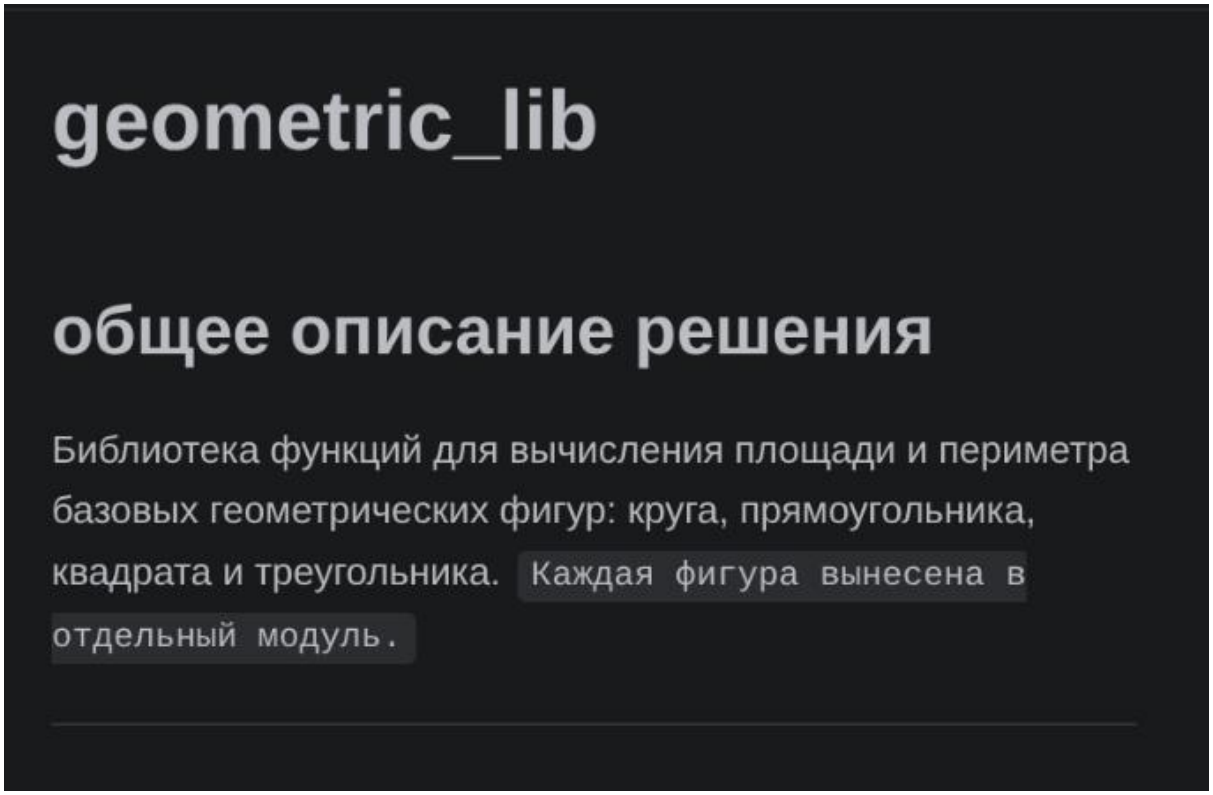

def perimeter(a :{__add__}, b, c):
    """
    Возвращает периметр треугольника.

        Параметры:
            a (int | float) : длина первой стороны треугольника
            b (int | float) : длина второй стороны треугольника
            c (int | float) : длина третьей стороны треугольника

        Возвращаемое значение:
            perimeter (float): периметр треугольника, вычисленный по формуле a + b + c

        Пример вывода
            perimeter(3, 4, 5) -> 12
    """
    return a + b + c
```

2. В каталоге docs/ создаем файл README.md в котором находится документация по проекту написанная в нотации markdown:



описание каждой функции с примерами вызова

circle.py

```
import math

def area(r):
    return math.pi * r * r

def perimeter(r):
    return 2 * math.pi * r
```

area(r):

- вычисление площади круга

функция принимает значение радиуса типа `int` или `float` и возвращает площадь круга типа `float` по формуле `pi * r * r`

```
# Пример вывода
area(3) # 28.274333882308138
```

perimeter(r):

- вычисление длины окружности круга

функция принимает значение радиуса типа `int` или `float` и возвращает длину окружности типа `float` по формуле `2 * pi * r`

```
# Пример вывода
perimeter(3) # 18.84955592153876
```

---

rectangle.py

```
def area(a, b):
    return a * b

def perimeter(a, b):
    return (a + b) * 2
```

area(a, b):

- вычисление площади прямоугольника

функция принимает значения двух сторон типа `int` или `float` и возвращает площадь прямоугольника типа `float` по формуле `a * b`

```
# Пример вывода
area(3, 4) # 12
```

perimeter(a, b):

- вычисление периметра прямоугольника

функция принимает значения двух сторон типа `int` или `float` и возвращает периметр прямоугольника типа `float` по формуле `(a + b) * 2`

```
# Пример вывода
perimeter(3, 4) # 14
```

---

square.py

```
def area(a):
    return a * a

def perimeter(a):
    return 4 * a
```

area(a):

- вычисление площади квадрата

функция принимает значение стороны типа `int` или `float` и возвращает площадь квадрата типа `float` по формуле `a * a`

```
# Пример вывода
area(5) # 25
```

perimeter(a):

- вычисление периметра квадрата

функция принимает значение стороны типа `int` или `float` и возвращает периметр квадрата типа `float` по формуле `4 * a`

```
# Пример вывода
perimeter(5) # 20
```

---

triangle.py

```
def area(a, h):
    return a * h / 2

def perimeter(a, b, c):
    return a + b + c
```

area(a, h):

- вычисление площади треугольника

функция принимает значения основания и высоты типа `int` или `float` и возвращает площадь треугольника типа `float` по формуле `a * h / 2`

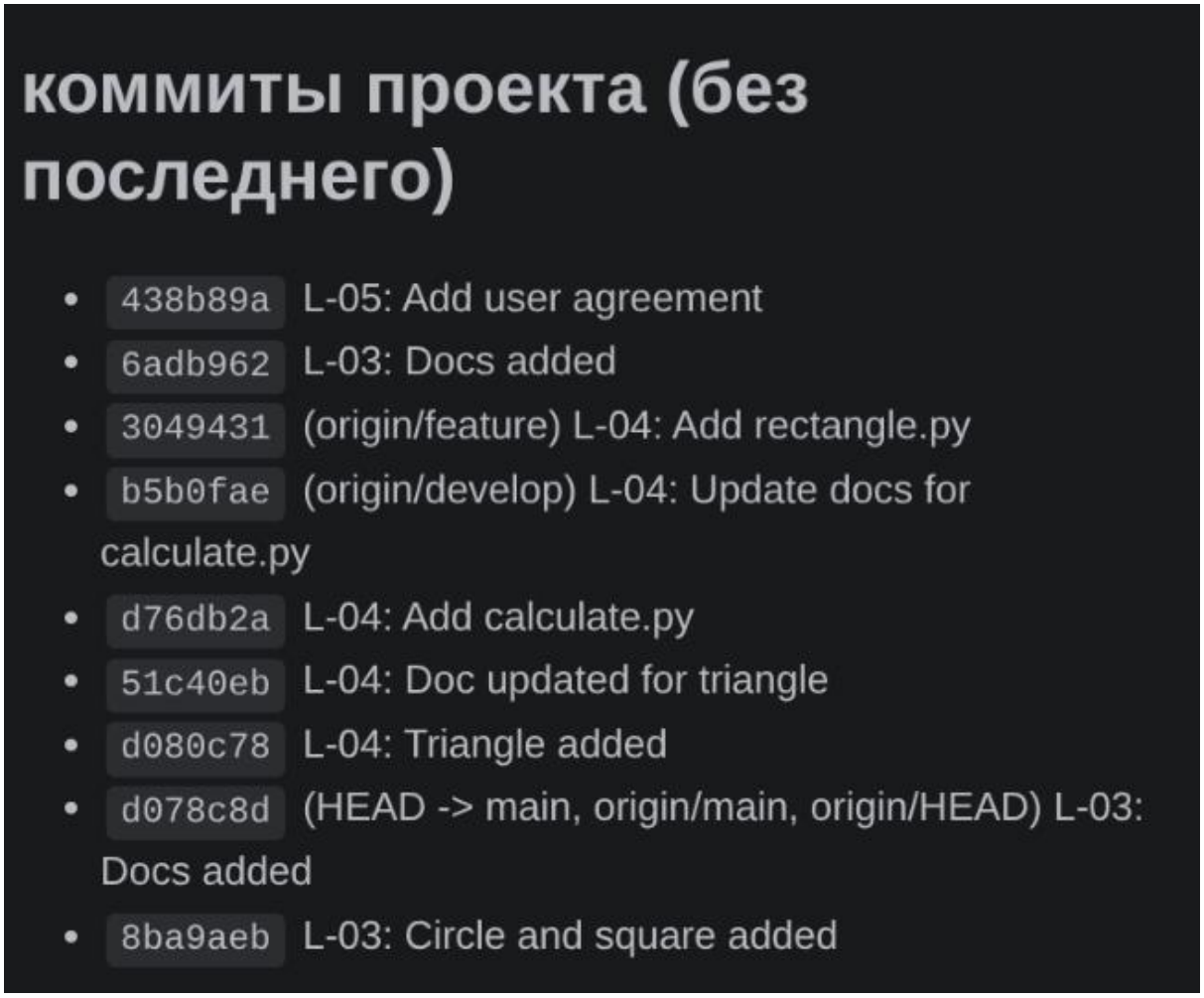
```
# Пример вывода
area(6, 4) # 12.0
```

perimeter(a, b, c):

- вычисление периметра треугольника

функция принимает значения трёх сторон типа `int` или `float` и возвращает периметр треугольника типа `float` по формуле `a + b + c`

```
# Пример вывода
perimeter(3, 4, 5) # 12
```



В конечном итоге для оформление текста документации в нотации markdown были использованы следующие конструкции:

- главный заголовок проекта (# geometric_lib)

- заголовки трех обязательных разделов задания

1. `## общее описание решения`
2. `## описание каждой функции с примерами вызова`
3. `## история изменения проекта`

`### - заголовок каждого файла внутри раздела с функциями`

1. `### circle.py`
2. `### rectangle.py`
3. `### square.py`
4. `### triangle.py`

- заголовок каждой отдельной функции внутри модуля

1. #### area(r)

2. #### perimeter(r):

[текст](ссылка) - кликабельная ссылка с названия файла на сам файл

1. [circle.py](../circle.py)

2. [rectangle.py](../rectangle.py)

3. [square.py](../square.py)

4. [triangle.py](../triangle.py)

``` python… ``` - блок кода с подсветкой синтаксиса, для показа кода функций и примеров их вызова

` … ` выделение формулы/значения
```